Implementation and Evaluation of a Model Transfer and Integration Workflow for Distributed Spiking Neural Network Learning

Hayato Takimoto s1300183 Supervised by Khanh N. Dang

Abstract

EnsembleSTDP trains Spiking Neural Network (SNN) sub-models on separate data subsets and integrates them at a centralized node. This study implements and evaluates a model-transfer and receiver-side integration workflow using TCP, ZeroMQ, and gRPC. Experiments examined workflow feasibility, sender execution timing, receiver-side bottlenecks, and final model accuracy. Concurrent execution reduced the mean global receiver-side span by 78.37–88.64%. In the nine-sender symmetric experiment, MNIST evaluation dominated the measured workflow time. Replacing a low-performing sub-model improved accuracy from 0.5719 to 0.8232. These results show that workflow performance depends on communication, sender timing, receiver-side processing, and sub-model quality. The source code and demonstration video are available at https://github.com/klab-aizu/GT_Protocols-for-EnsembleSTDP.

1 Introduction

Spiking Neural Networks (SNNs) process information using discrete spike events and have been studied as biologically inspired models for event-driven and energy-efficient computing [1]. Unlike conventional artificial neural networks, which generally transmit continuous-valued activations, SNNs transmit information through the timing and frequency of spikes. A spiking neuron integrates incoming signals over time and emits a spike when its membrane potential reaches a firing threshold.

The SNN used by the existing EnsembleSTDP implementation is based on the Diehl and Cook model provided by BindsNET [1, 2]. As shown in Figure 1, each MNIST image has a resolution of 28×28 pixels and is converted into 784 input spike trains using Poisson encoding. The input layer is fully connected to an excitatory layer whose number of neurons depends on the sub-model configuration. Synaptic weights between the input and excitatory layers are updated through Spike-Timing-Dependent Plasticity (STDP). Each excitatory neuron is connected to a corresponding inhibitory neuron, and the inhibitory neurons suppress competing excitatory neurons. This lateral inhibition encourages different excitatory neurons to respond to different input patterns.

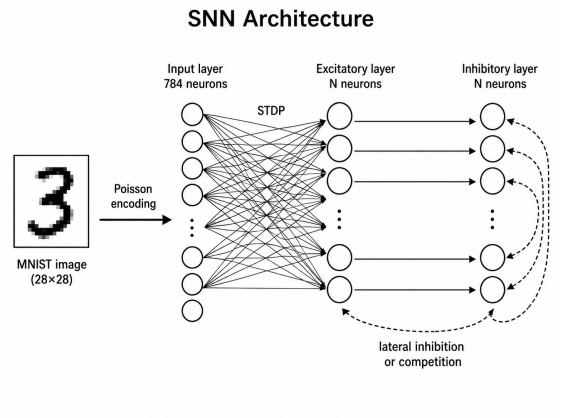

Figure 1: SNN architecture used in the existing EnsembleSTDP implementation. An MNIST image is converted into 784 spike trains using Poisson encoding. The input layer is fully connected to the excitatory layer through STDP-enabled synapses. Each excitatory neuron is connected to a corresponding inhibitory neuron, and the inhibitory neurons suppress the other excitatory neurons.

Spike-Timing-Dependent Plasticity updates synaptic weights according to the relative timing of pre-synaptic and post-synaptic spikes [3]. A commonly used pair-based exponential STDP model can be expressed as [4]

$$\Delta t = t_{\text{post}} - t_{\text{pre}}, \tag{1}$$

$$\Delta w = \begin{cases} A_+ \exp\left(-\Delta t/\tau_+\right), & \Delta t > 0, \\ -A_- \exp\left(\Delta t/\tau_-\right), & \Delta t < 0. \end{cases} \tag{2}$$

Here, t_{pre} and t_{post} denote the pre-synaptic and post-synaptic spike times, respectively. The parameters A_+ and A_- determine the maximum amounts of potentiation and depression, while τ_+ and τ_- are their time constants. When a pre-synaptic spike occurs before a post-synaptic spike ($\Delta t > 0$), the synaptic connection is strengthened. When the order is reversed ($\Delta t < 0$), the connection is weakened. Because this learning rule uses locally available spike-timing information, it is suitable for on-device and neuromorphic learning. Figure 2

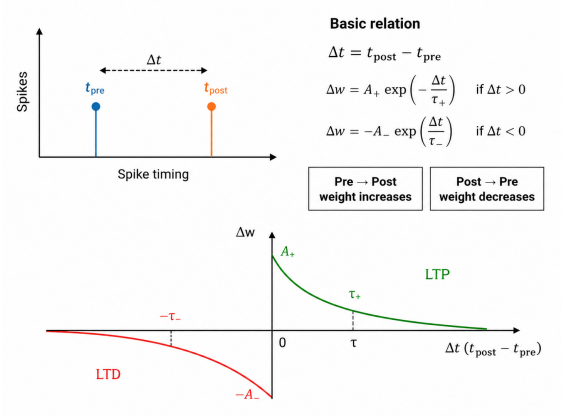

Figure 2: Basic concept of Spike-Timing-Dependent Plasticity. A positive timing difference produces potentiation, whereas a negative timing difference produces depression.

illustrates the basic timing relationship and the corresponding STDP learning window.

EnsembleSTDP divides a training dataset into multiple subsets and trains independent SNN sub-models on the subsets [5]. After local training, the sub-models are collected at a centralized node, merged, and compressed by removing redundant neurons. This approach can distribute the training workload while maintaining a single integrated model.

However, applying EnsembleSTDP in a distributed environment requires more than the existing learning and merging procedures. Multiple sender nodes must transfer complete model-related file sets to a receiver. The receiver must store the files, verify successful reception, preserve the correspondence among the files, record timing information, and prepare the files for model integration. The completion time may also be affected by sender execution mode, delayed senders, differences in arrival time, receiver-side processing, and model quality.

The purpose of this study is to implement and evaluate the model-transfer and receiver-side integration workflow required for distributed SNN learning based on EnsembleSTDP. The underlying STDP training, model-merging, and neuron-compression procedures are derived from the existing EnsembleSTDP implementation. This study focuses on communication, automatic sender execution, file verification, receiver-side logging, file aggregation, and system-level performance evaluation.

TCP, ZeroMQ, and gRPC were implemented as communication approaches. The workflow was evaluated using symmetric and asymmetric sub-model configurations. Additional experiments compared sequential

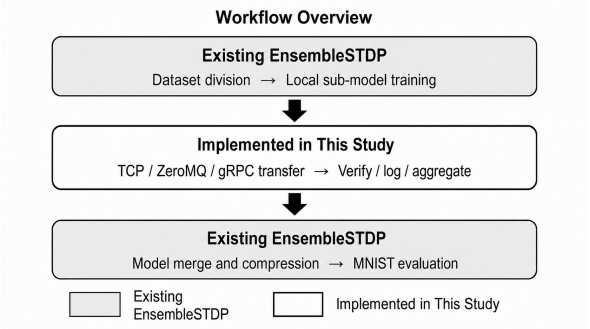

Figure 3: Overall workflow and research scope. Gray boxes represent the existing EnsembleSTDP procedures, whereas the white box represents the model transfer and receiver-side processing implemented in this study.

and concurrent sender execution, introduced an artificial sender delay, analyzed receiver-side arrival imbalance, divided the workflow into processing stages, and examined the relationship between sub-model quality and final integrated accuracy.

The remainder of this GT is organized as follows. Section 1 introduces SNNs, STDP, EnsembleSTDP, and the motivation and objective of this study. Section 2 presents the model-transfer and receiver-side integration workflow, including the implemented communication approaches. Section 3 describes the evaluation objectives, experimental setup, timing metrics, results, discussion, and limitations. Section 4 concludes this study and outlines future work.

2 Proposal

2.1 Overall Workflow and Research Scope

Figure 3 shows the overall workflow and the scope of this study. The existing EnsembleSTDP procedures include dataset division, local sub-model training, model merging, neuron compression, and final model evaluation.

This study focuses on the intermediate model-transfer and receiver-side processing stage. Trained sub-models are transferred from multiple sender nodes, verified and stored at the receiver, and prepared for the existing integration procedure.

2.2 EnsembleSTDP Files and Integration

Each trained sub-model consists of three related files:

- `model_*.pth`, containing the trained network parameters;

- `assignments_*.pth`, containing the label assigned to each neuron; and
- `proportions_*.pth`, containing class-response information used during classification.

All three files are required for receiver-side merging and evaluation. After the files are transferred, the receiver collects them in the integration directory. The existing EnsembleSTDP procedure then concatenates the sub-models, identifies similar neurons using Mean Squared Error (MSE), removes the specified number of neurons, and evaluates the final model on MNIST [5,6].

2.3 Transfer and Integration Workflow

One Ubuntu virtual machine, `vm-node1`, was used as the receiver. The remaining virtual machines were used as senders. The largest configuration used nine sender virtual machines, `vm-node2` through `vm-node10`. Each sender transferred one sub-model consisting of three files. Therefore, the nine-sender configuration transferred 27 files in total.

The receiver performed the following operations:

1. received model-related files from the sender nodes;
2. stored the files in protocol-specific directories;
3. checked the expected number of received files;
4. recorded receiver-side timestamps;
5. performed integrity verification when supported by the implementation;
6. aggregated the files into the integration directory; and
7. executed merge, compression, and MNIST evaluation.

2.4 Communication and Automatic Execution

The TCP implementation used Python sockets. The sender transmitted file metadata and file data to the receiver and waited for acknowledgement. SHA-256 values were used to confirm file integrity. The sender program originally supported a fixed interval between file transfers. For the reported communication comparisons, this interval was set to zero.

The ZeroMQ implementation used a request–reply communication pattern. Each sender transferred file metadata and file data, and the receiver recorded the reception time of each file.

The gRPC implementation used Protocol Buffers and a client-streaming Remote Procedure Call. Files were divided into chunks and transmitted as a stream. The receiver reconstructed and stored each file and performed integrity verification.

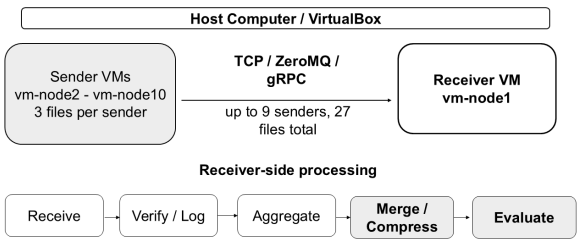

Figure 4: Experimental setup and receiver-side processing. The largest configuration used nine sender virtual machines and one receiver virtual machine, with three files transferred per sender.

Shell scripts were prepared to reduce manual operation. They started the receiver, connected to sender virtual machines through SSH, prepared the corresponding files, executed sender programs, and checked the received file count. Separate execution scripts were used for sequential execution, concurrent execution, and artificial sender-delay experiments.

2.5 Sub-model and Quality Evaluation

In this study, one sub-model is one SNN trained independently on a subset of MNIST. Sub-model quality is evaluated using the standalone MNIST classification accuracy of each sub-model before integration.

When a low-performing sub-model is identified, its effect is examined by removing or replacing it and evaluating the integrated model again. This procedure is used to distinguish the effect of the communication approach from the effect of the sub-model set itself.

3 Evaluation

3.1 Evaluation Objectives

The evaluation was organized around four objectives:

1. **Workflow feasibility:** confirm that TCP, ZeroMQ, and gRPC can successfully transfer the required files and support receiver-side integration;
2. **Workflow completion time:** evaluate the effects of sequential and concurrent sender execution, a delayed sender, and sender arrival imbalance;

Table 1: Sub-model configurations.

Configuration	Before	Removed	Final
3-sender asym.	350	50	300
9-sender sym.	900	100	800
9-sender asym.	1350	350	1000

3. **Receiver-side processing:** identify the dominant processing stage in the measured workflow; and

4. **Final integrated accuracy:** examine the effects of the communication approach and sub-model quality on the integrated model.

3.2 Experimental Configurations

Three main sub-model configurations were evaluated. The three-sender asymmetric configuration contained 200-, 100-, and 50-neuron sub-models. The nine-sender symmetric configuration contained nine 100-neuron sub-models. The nine-sender asymmetric configuration contained sub-models ranging from 250 to 50 neurons.

The sequential and concurrent execution experiments used seven senders. Each sender transferred three files, resulting in 21 files per trial. Ten trials were conducted for each communication approach and execution mode.

The controlled-straggler experiment used ZeroMQ and artificially delayed one sender by 0.5, 1, 2, or 5 s. The bottleneck experiment used the nine-sender symmetric configuration and measured transfer, merge and compression, and MNIST evaluation separately.

3.3 Receiver-Side Timing Metrics

Some early transfer experiments started sender nodes manually. Therefore, the raw batch duration included idle time between sender executions. To remove this manual idle time, corrected receiver-side transfer time was calculated as

$$T_{\text{corrected}} = \sum_{s=1}^{N} \left(t_{\text{last},s} - t_{\text{first},s} \right), \tag{3}$$

where $t_{\text{first},s}$ is the first receive-completion timestamp among the three files transferred by sender s, and $t_{\text{last},s}$ is the last receive-completion timestamp for that sender.

For concurrent execution, the global receiver-side span was calculated as

$$T_{\text{global}} = \max_{s} \left(t_{\text{last},s} \right) - \min_{s} \left(t_{\text{first},s} \right). \tag{4}$$

Sender arrival skew was calculated as

$$T_{\text{arrival}} = \max_{s} \left(t_{\text{first},s} \right) - \min_{s} \left(t_{\text{first},s} \right). \tag{5}$$

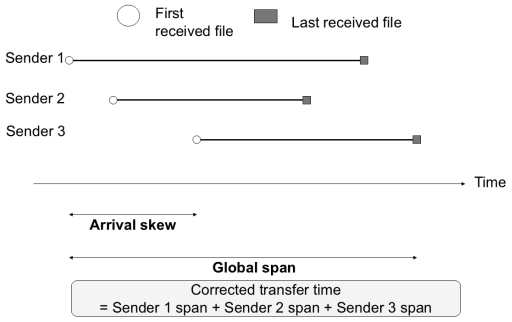

Figure 5: Definitions of corrected receiver-side transfer time, sender arrival skew, and global receiver-side span.

Table 2: Final integrated model accuracy.

Configuration	Final neurons	Accuracy
3-sender asym.	300	0.8676
9-sender sym.	800	0.8800
9-sender asym., initial	1000	0.5719
9-sender asym., improved	1000	0.8232

Corrected transfer time represents the sum of the individual sender reception spans. Global span represents the interval from the earliest sender reception to the completion of the final sender. Arrival skew represents the difference between the earliest and latest sender arrival times.

These values are receiver-side measurements. They are not pure network latency or direct measurements of synchronization overhead.

3.4 Result 1: Workflow Validation and Model Integration

TCP, ZeroMQ, and gRPC successfully transferred the required model-related files and supported receiver-side integration in the tested configurations. When the same sub-model files were transferred correctly, all three communication approaches produced the same final integrated accuracy.

In the three-sender asymmetric experiment, the mean corrected receiver-side transfer times were 0.355 s for TCP, 0.130 s for ZeroMQ, and 0.291 s for gRPC. All three approaches produced a final model containing 300 neurons and an accuracy of 0.8676.

The nine-sender symmetric configuration produced a final model containing 800 neurons and an accuracy of 0.8800. The initial nine-sender asymmetric configuration produced a final model containing 1,000 neurons but achieved an accuracy of only 0.5719.

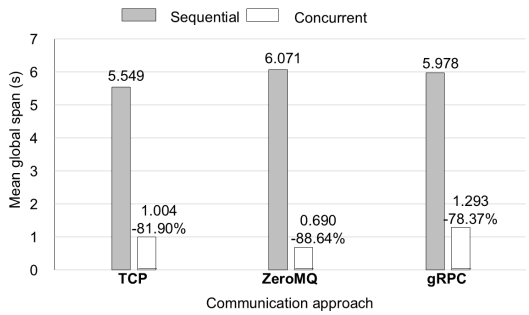

Figure 6: Mean global receiver-side span for sequential and concurrent sender execution. Ten trials were conducted for each communication approach and execution mode.

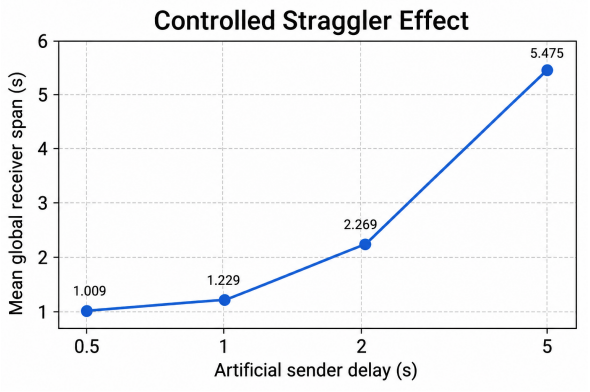

Figure 7: Effect of artificial sender delay on the mean global receiver-side span.

Table 3: Receiver-side timing during concurrent execution.

Approach	Sender span (s)	Arrival skew (s)	Global span (s)
TCP	0.093	0.853	1.004
ZeroMQ	0.086	0.601	0.690
gRPC	0.070	1.193	1.293

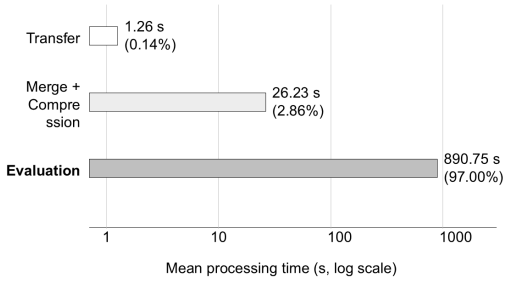

Figure 8: Mean processing times and approximate proportions of the measured workflow stages in the nine-sender symmetric configuration.

3.5 Result 2: Sender Execution and Timing

3.5.1 Sequential and Concurrent Execution

Figure 6 compares the mean global receiver-side span for sequential and concurrent sender execution.

Concurrent execution reduced the mean global receiver-side span by 81.90% for TCP, 88.64% for ZeroMQ, and 78.37% for gRPC. The reduction was observed for all three approaches, indicating that overlapping sender transfer periods were more effective than executing the senders one at a time.

3.5.2 Straggler and Arrival Imbalance

The controlled-straggler experiment examined the effect of delaying one sender. Artificial delays of 0.5, 1, 2, and 5 s produced mean global receiver-side spans of 1.009, 1.229, 2.269, and 5.475 s, respectively.

As the artificial delay increased, the global completion span also increased. This result shows that the final arriving sender can determine the completion time of the full transfer stage.

The concurrent receiver-side timing analysis also showed differences among sender arrival times. Table 3 summarizes the mean sender span, arrival skew, and global span during concurrent execution.

gRPC had the shortest mean sender span of 0.070 s but the largest mean arrival skew of 1.193 s. ZeroMQ had the shortest mean global span of 0.690 s. Therefore, short individual sender spans did not necessarily produce a short global completion time.

3.6 Result 3: Bottleneck Breakdown

In the nine-sender symmetric configuration, the mean corrected receiver-side transfer times were 1.233 s for TCP, 1.356 s for ZeroMQ, and 1.203 s for gRPC. These differences were small compared with the later workflow stages.

Across four complete executions, mean merge and compression time was 26.23 s and mean MNIST evaluation time was 890.75 s. The average transfer time across the three communication approaches was approximately 1.26 s.

The approximate proportions of the measured stage time were 0.14% for transfer, 2.86% for merge and com-

pression, and 97.00% for evaluation. Therefore, MNIST evaluation was the dominant stage under the tested conditions.

3.7 Result 4: Effect of Sub-model Quality

The low accuracy of the initial nine-sender asymmetric model was investigated by evaluating its constituent sub-models. One 250-neuron sub-model achieved an individual accuracy of 0.5349, which was substantially lower than the accuracies of the other sub-models.

Removing the low-performing sub-model increased the integrated accuracy from 0.5719 to 0.8033. After replacing the low-performing sub-model, the integrated accuracy further increased to 0.8232. The final improved model contained 1,000 neurons.

This result suggests that the low-performing sub-model was a major contributor to the accuracy degradation. However, the experiment does not prove that it was the only cause.

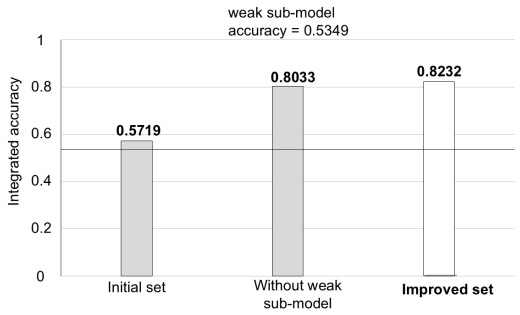

Figure 9: Effect of sub-model quality on the accuracy of the nine-sender asymmetric integrated model.

3.8 Discussion

3.8.1 Factors Affecting Workflow Completion Time

The experimental results show that the communication approach alone did not determine workflow completion time. The relative performance of TCP, ZeroMQ, and gRPC changed depending on the model configuration, sender execution mode, and timing metric.

Concurrent execution substantially reduced the global span for all three communication approaches. In sequential execution, the sender spans were accumulated one after another. In concurrent execution, these spans overlapped, reducing the overall interval observed at the receiver.

However, concurrent execution did not eliminate sender imbalance. The controlled-straggler experiment showed that one delayed sender increased the completion time of the entire transfer stage. The arrival-skew analysis also showed that a communication approach could have short per-sender reception spans while still producing a longer global span because the senders did not arrive at the receiver simultaneously.

The bottleneck experiment showed that optimizing file transfer alone would have a limited effect on the measured end-to-end workflow under the tested conditions. Transfer required approximately 1.2–1.4 s, whereas evaluation required approximately 891 s. Therefore, reducing downstream evaluation cost may produce a larger overall improvement than reducing the relatively small differences among the communication approaches.

3.8.2 Communication Approach and Model Accuracy

Communication performance and model accuracy should be evaluated separately. When the same sub-model files were transferred correctly, TCP, ZeroMQ, and gRPC produced the same final accuracy. This indicates that the communication approach did not change the model result in these experiments after successful file reception.

In contrast, the quality of the sub-model set had a substantial effect on the integrated model. The initial asymmetric model set included a low-performing 250-neuron sub-model and achieved an integrated accuracy of 0.5719. Removing the weak sub-model increased the accuracy to 0.8033, while replacing it increased the accuracy to 0.8232.

Therefore, successful transfer and integration do not by themselves guarantee a high-quality final model. Distributed EnsembleSTDP evaluation should consider both system-level communication performance and learning-level sub-model quality.

3.8.3 Limitations

The experiments were conducted using virtual machines in VirtualBox. The measurements may therefore include the effects of host scheduling, virtualized networking, Python execution, disk input/output, and competition for physical resources.

Corrected receiver-side transfer time is not pure network latency. It also includes receiver-side protocol processing, file storage, logging, and operating-system effects. Similarly, arrival skew is an observed difference in sender arrival timing and is not a direct measurement of synchronization overhead.

The bottleneck proportions are specific to the BindsNET-based MNIST evaluation, 10,000 test images, and the virtual-machine environment used in this study. The measured workflow does not include the complete distributed training process. Therefore, the result that evaluation accounted for approximately 97% of the measured time should not be generalized to all

distributed SNN systems.

4 Conclusion

This study implemented and evaluated a model-transfer and receiver-side integration workflow for distributed SNN learning based on EnsembleSTDP using TCP, ZeroMQ, and gRPC. All three communication approaches successfully supported model transfer and integration under symmetric and asymmetric configurations, while concurrent execution reduced completion time and sender delay and arrival imbalance increased it. Under the tested conditions, MNIST evaluation dominated the measured workflow time, and final integrated accuracy was affected more strongly by sub-model quality than by the communication approach. These results show that communication, sender execution timing, receiver-side processing, and learning quality should be evaluated together, and future work should evaluate the complete workflow on physical heterogeneous devices and investigate methods for reducing sender imbalance and evaluation cost.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Khanh N. Dang, for his guidance and support throughout this research. I also thank the members of the laboratory for their advice and assistance.

References

[1] P. U. Diehl and M. Cook, "Unsupervised Learning of Digit Recognition Using Spike-Timing-Dependent Plasticity," *Frontiers in Computational Neuroscience*, vol. 9, Art. no. 99, 2015, doi: 10.3389/fncom.2015.00099.

[2] H. Hazan, D. J. Saunders, H. Khan, D. Patel, D. T. Sanghavi, H. T. Siegelmann, and R. Kozma, "BindsNET: A Machine Learning-Oriented Spiking Neural Networks Library in Python," *Frontiers in Neuroinformatics*, vol. 12, Art. no. 89, 2018, doi: 10.3389/fninf.2018.00089.

[3] G.-Q. Bi and M.-M. Poo, "Synaptic Modifications in Cultured Hippocampal Neurons: Dependence on Spike Timing, Synaptic Strength, and Postsynaptic Cell Type," *The Journal of Neuroscience*, vol. 18, no. 24, pp. 10464–10472, 1998, doi: 10.1523/JNEUROSCI.18-24-10464.1998.

[4] S. Song, K. D. Miller, and L. F. Abbott, "Competitive Hebbian Learning Through Spike-Timing-Dependent Synaptic Plasticity," *Nature Neuroscience*, vol. 3, no. 9, pp. 919–926, 2000, doi: 10.1038/78829.

[5] Y. Hanyu and K. N. Dang, "EnsembleSTDP: Distributed in-situ Spike Timing Dependent Plasticity Learning in Spiking Neural Networks," in *Proceedings of the 2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, 2024, pp. 434–440, doi: 10.1109/MCSoC64144.2024.00077.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791.